

pandas

Pandas for Your Thesis¶

A practical, focused guide — exactly what you need for *Benchmarking Tabular Data Augmentation Techniques for Deep Clustering*.

You already know **numpy**. Think of pandas as **numpy with labels**:

- A **Series** is a 1-D numpy array whose rows have **an index** (labels).
- A **DataFrame** is a 2-D table whose rows have an index **and** whose columns have **names**.

That's the whole mental model. Almost every pandas operation is just numpy + labels.

What this notebook covers (everything you'll actually use):

1. Loading data — CSV, sklearn, OpenML
2. Inspecting a dataset
3. Selecting rows and columns
4. Numerical vs categorical columns
5. Handling missing values
6. Simple transformations
7. Splitting features (X) from labels (y)
8. Handing data over to scikit-learn / PyTorch
9. Saving results to CSV
10. What you don't need to learn for *this* thesis
11. A full end-to-end mini-example
12. Quick reference card

Run each code cell with **Shift + Enter**. Read it, modify it, run it again — that's how pandas sticks.

0. Setup¶

In [1]:

```
import numpy as np
import pandas as pd
```

```
print("pandas:", pd.__version__)
print("numpy :", np.__version__)
```

```
pandas: 2.2.3
```

```
numpy : 2.2.3
```

1. Loading data¶

Three patterns cover almost everything you'll do in this thesis.

(a) From a CSV file — most common:

```
df = pd.read_csv("path/to/file.csv")
```

Useful arguments:

- `sep=","` — separator (use `;"` for semicolon, `"\t"` for tab)
- `header=0` — which row is the header (`None` if there is no header)
- `na_values=["?", "NA", ""]` — strings that should be treated as missing
- `usecols=["a", "b"]` — load only some columns
- `dtype={"col": str}` — force a column type

(b) From scikit-learn's built-in datasets (no download needed):

```
from sklearn.datasets import load_iris
data = load_iris(as_frame=True)
df = data.frame      # full DataFrame
X = data.data        # features as DataFrame
y = data.target      # labels as Series
```

(c) From OpenML (lots of tabular datasets with ground-truth labels — perfect for your benchmark):

```
from sklearn.datasets import fetch_openml
data = fetch_openml("adult", version=2, as_frame=True)
df = data.frame
```

In [2]:

```
# Load Iris as our practice dataset
from sklearn.datasets import load_iris
```

```
iris = load_iris(as_frame=True)
df = iris.frame.copy()
df.head()
```

Out[2]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

For the parts where we need **categorical columns** (Iris is all numerical), we'll also build a small synthetic dataset:

In [3]:

```
rng = np.random.default_rng(seed=42)
n = 8
small = pd.DataFrame({
    "age":    rng.integers(18, 70, size=n),
    "income": rng.normal(40000, 15000, size=n).round(),
    "city":   rng.choice(["Berlin", "Munich", "Hamburg"], size=n),
    "gender": rng.choice(["F", "M"], size=n),
    "label":  rng.integers(0, 2, size=n),
})
# Inject a missing value to practise on
small.loc[2, "income"] = np.nan
small
```

Out[3]:

	age	income	city	gender	label
0	22	10734.0	Hamburg	F	0
1	58	20467.0	Munich	M	1
2	52	NaN	Munich	M	1
3	40	35256.0	Hamburg	F	0
4	40	39748.0	Munich	M	0
5	62	27204.0	Munich	M	1
6	22	53191.0	Munich	F	0
7	54	51667.0	Berlin	M	1

2. Inspecting a dataset ¶

The first thing you should do with any new dataset. These five lines tell you 80% of what you need to know.

In [4]:

```
print("Shape:", df.shape)          # (rows, cols)
print("Columns:", list(df.columns))
```

```

print()
df.info()                                # types + non-null counts per column

Shape: (150, 5)
Columns: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)',

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
4   target                 150 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB

```

In [5]:

```
df.head()          # first 5 rows  (df.tail() for the last 5)
```

Out[5]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

In [6]:

```
df.describe()      # numerical summary: mean, std, min, max, quartiles
```

Out[6]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
max	7.900000	4.400000	6.900000	2.500000	2.000000

In [7]:

```
df.dtypes          # type of each column
```

Out[7]:

```
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target              int64
dtype: object
```

In [8]:

```
df.isna().sum()    # how many missing values per column
```

Out[8]:

```
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target              0
dtype: int64
```

In [9]:

```
df["target"].value_counts()    # class balance - important for clustering!
```

Out[9]:

```
target
0      50
1      50
2      50
Name: count, dtype: int64
```

Exercise 1a — Loading & first look¶

Load the **wine** dataset from scikit-learn as a DataFrame. Print its shape and display the first 3 rows.

Hint: from sklearn.datasets import load_wine — use as_frame=True, then .frame.

In [10]:

```
# Exercise 1a
from sklearn.datasets import load_wine

# Load as a DataFrame
df_wine = load_wine(as_frame=True).frame.copy()

# Print shape
print("Wine shape:", df.shape)

# Show first 3 rows
df_wine.head(3)

Wine shape: (150, 5)

Out[10]:
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoids
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30

```
In [11]:

# --- Solution (uncomment to reveal) ---
# wine_data = load_wine(as_frame=True)
# df_wine = wine_data.frame
# print('Shape:', df_wine.shape)
# df_wine.head(3)
```

Exercise 1b — Inspecting missing values & class balance¶

Using `df_wine` from Exercise 1a:

1. Print how many missing values each column has.
2. Print the class balance of the `target` column.

```
In [12]:

# Exercise 1b (assumes df_wine is defined from 1a)
# 1. Missing values per column
print(df_wine.isna().sum())

# 2. Class balance of target
df_wine["target"].value_counts()

alcohol          0
malic_acid       0
```

```

ash                                0
alkalinity_of_ash                  0
magnesium                          0
total_phenols                      0
flavanoids                        0
nonflavanoid_phenols              0
proanthocyanins                   0
color_intensity                   0
hue                                0
od280/od315_of_diluted_wines     0
proline                           0
target                            0
dtype: int64

```

Out[12]:

```

target
1      71
0      59
2      48
Name: count, dtype: int64

```

In [13]:

```

# --- Solution ---
# print(df_wine.isna().sum())
# df_wine['target'].value_counts()

```

3. Selecting rows and columns¶

There are three main ways. Knowing all three is enough for everything in your thesis.

In [14]:

```

# Single column → Series (1-D)
df["sepal length (cm)"].head()

```

Out[14]:

```

0      5.1
1      4.9
2      4.7
3      4.6
4      5.0
Name: sepal length (cm), dtype: float64

```

In [15]:

```

# Multiple columns → DataFrame
df[["sepal length (cm)", "sepal width (cm)"]].head()

```

Out[15]:

	sepal length (cm)	sepal width (cm)
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2
3	4.6	3.1
4	5.0	3.6

In [16]:

```
# Rows by POSITION (like numpy indexing)
df.iloc[0]          # first row
df.iloc[0:5]        # first five rows
df.iloc[0:5, 0:2]    # first five rows, first two columns
```

Out[16]:

	sepal length (cm)	sepal width (cm)
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2
3	4.6	3.1
4	5.0	3.6

In [17]:

```
# Rows by LABEL (using the index)
df.loc[0]          # row with index label 0
df.loc[0:5, "sepal length (cm)"]  # rows 0..5, one column
```

Out[17]:

```
0    5.1
1    4.9
2    4.7
3    4.6
4    5.0
5    5.4
Name: sepal length (cm), dtype: float64
```

In [18]:

```
# Boolean filtering - extremely common
mask = df["sepal length (cm)"] > 6
df[mask].head()
```

Out[18]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
54	6.5	2.8	4.6	1.5	1
56	6.3	3.3	4.7	1.6	1

Exercise 2a — Selecting multiple columns¶

From `df`, select only "petal length (cm)" and "petal width (cm)" and display the first 5 rows.

In [19]:

```
# Exercise 2a
# your code here
df[["petal length (cm)", "petal width (cm)"]].head()
```

Out[19]:

	petal length (cm)	petal width (cm)
0	1.4	0.2
1	1.4	0.2
2	1.3	0.2
3	1.5	0.2
4	1.4	0.2

In [20]:

```
# --- Solution ---
# df[['petal length (cm)', 'petal width (cm)']].head()
```

Exercise 2b — Boolean filtering¶

Create a boolean mask for rows where "sepal width (cm)" is less than 2.5, apply it to `df`, and print how many rows match.

In [21]:

```
# Exercise 2b
mask = df['sepal width (cm)'] < 2.5 # your code here
filtered = df[mask] # your code here
print('Rows matching:', len(filtered))
filtered.head()
```

Rows matching: 11

Out[21]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
41	4.5	2.3	1.3	0.3	0
53	5.5	2.3	4.0	1.3	1
57	4.9	2.4	3.3	1.0	1
60	5.0	2.0	3.5	1.0	1
62	6.0	2.2	4.0	1.0	1

In [22]:

```
# --- Solution ---
# mask = df['sepal width (cm)'] < 2.5
# filtered = df[mask]
# print('Rows matching:', len(filtered))
# filtered.head()
```

Exercise 2c — Positional indexing with `.iloc`

Use `.iloc` to select rows **10 through 20** (inclusive) and columns **0 and 2** only.

In [23]:

```
# Exercise 2c
# your code here
df.iloc[10:21, [0,2]]
```

Out[23]:

	sepal length (cm)	petal length (cm)
10	5.4	1.5
11	4.8	1.6
12	4.8	1.4
13	4.3	1.1
14	5.8	1.2
15	5.7	1.5
16	5.4	1.3
17	5.1	1.4
18	5.7	1.7
19	5.1	1.5
20	5.4	1.7

In [24]:

```
# --- Solution ---
# df.iloc[10:21, [0, 2]]
```

4. Numerical vs categorical columns¶

In your thesis, **numerical** columns are scaled (e.g. `StandardScaler`), **categorical** columns are encoded (e.g. `OneHotEncoder`). Pandas helps you separate them in one line:

In [25]:

```
num_cols = small.select_dtypes(include="number").columns.tolist()
cat_cols = small.select_dtypes(include=["object", "category"]).columns.tolist()

print("Numerical :", num_cols)
print("Categorical:", cat_cols)

Numerical : ['age', 'income', 'label']
Categorical: ['city', 'gender']
```

Exercise 3a — Separating column types¶

Using `small`, build:

- `num_cols_s` — all numerical column names
- `cat_cols_s` — all categorical (object/category) column names

Print both lists.

In [26]:

```
# Exercise 3a
num_cols_s = small.select_dtypes(include='number').columns.tolist() # your code here
cat_cols_s = small.select_dtypes(include=['object', 'category']).columns.tolist() # your code here
print('Numerical :', num_cols_s)
print('Categorical:', cat_cols_s)

Numerical : ['age', 'income', 'label']
Categorical: ['city', 'gender']
```

In [27]:

```
# --- Solution ---
# num_cols_s = small.select_dtypes(include='number').columns.tolist()
# cat_cols_s = small.select_dtypes(include=['object', 'category']).columns.tolist()
# print('Numerical :', num_cols_s)
# print('Categorical:', cat_cols_s)
```

Exercise 3b — Adding a column and re-inspecting types¶

Add a column "size" to `small` with random values from ["small", "medium", "large"]. Then re-run `select_dtypes` and confirm "size" appears in the categorical list.

Hint: `rng.choice(["small", "medium", "large"], size=len(small))`

In [28]:

```
# Exercise 3b
small['size'] = rng.choice(['small', 'medium', 'large'], size=len(small))    # your code here

# Re-check categorical columns
cat_cols_s = small.select_dtypes(include=['object', 'category']).columns.tolist()    # your code here
print('Categorical now:', cat_cols_s)

Categorical now: ['city', 'gender', 'size']
```

In [29]:

```
# --- Solution ---
# small['size'] = rng.choice(['small', 'medium', 'large'], size=len(small))
# cat_cols_s = small.select_dtypes(include=['object', 'category']).columns.tolist()
# print('Categorical now:', cat_cols_s)
```

5. Handling missing values¶

Real datasets have gaps. Two strategies cover most cases.

In [30]:

```
small.isna().sum()    # how many missing per column
```

Out[30]:

```
age      0
income   1
city      0
gender    0
label     0
size      0
dtype: int64
```

In [31]:

```
# Option A: drop rows that have any missing value
small.dropna()
```

Out[31]:

	age	income	city	gender	label	size
0	22	10734.0	Hamburg	F	0	large
1	58	20467.0	Munich	M	1	large
3	40	35256.0	Hamburg	F	0	small
4	40	39748.0	Munich	M	0	medium
5	62	27204.0	Munich	M	1	medium
6	22	53191.0	Munich	F	0	medium
7	54	51667.0	Berlin	M	1	small

In [32]:

```
# Option B: fill with a value (mean is a common default for numerical columns)
small_filled = small.copy()
small_filled["income"] = small_filled["income"].fillna(small_filled["income"].mean())
small_filled
```

Out[32]:

	age	income	city	gender	label	size
0	22	10734.000000	Hamburg	F	0	large
1	58	20467.000000	Munich	M	1	large
2	52	34038.142857	Munich	M	1	large
3	40	35256.000000	Hamburg	F	0	small
4	40	39748.000000	Munich	M	0	medium
5	62	27204.000000	Munich	M	1	medium
6	22	53191.000000	Munich	F	0	medium
7	54	51667.000000	Berlin	M	1	small

Exercise 4a — Filling with the median¶

Make a copy of `small` called `small_med`. Fill the missing `"income"` value with the **median** of that column (not the mean). Confirm there are no more NaNs.

In [33]:

```
# Exercise 4a
small_med = small.copy()
# Fill missing income with median
# your code here
small_med['income'] = small_med['income'].fillna(small_med['income'].median())
print(small_med.isna().sum())
small_med

age      0
income   0
```

```
city      0
gender    0
label     0
size      0
dtype: int64
```

Out[33]:

	age	income	city	gender	label	size
0	22	10734.0	Hamburg	F	0	large
1	58	20467.0	Munich	M	1	large
2	52	35256.0	Munich	M	1	large
3	40	35256.0	Hamburg	F	0	small
4	40	39748.0	Munich	M	0	medium
5	62	27204.0	Munich	M	1	medium
6	22	53191.0	Munich	F	0	medium
7	54	51667.0	Berlin	M	1	small

In [34]:

```
# --- Solution ---
# small_med = small.copy()
# small_med['income'] = small_med['income'].fillna(small_med['income'].median())
# print(small_med.isna().sum())
# small_med
```

Exercise 4b — Dropping rows with missing categoricals¶

Use `dropna(subset=...)` to drop any row in `small` where a **categorical** column has a missing value.

Hint: build `cat_cols_s` with `select_dtypes` first, then pass it as `subset`.

In [35]:

```
# Exercise 4b
cat_cols_s = small.select_dtypes(include=['object', 'category']).columns.tolist()
# Drop rows with NaN in any categorical column
# your code here
small.dropna(subset=cat_cols_s)
```

Out[35]:

	age	income	city	gender	label	size
0	22	10734.0	Hamburg	F	0	large
1	58	20467.0	Munich	M	1	large
2	52	NaN	Munich	M	1	large

	age	income	city	gender	label	size
3	40	35256.0	Hamburg	F	0	small
4	40	39748.0	Munich	M	0	medium
5	62	27204.0	Munich	M	1	medium
6	22	53191.0	Munich	F	0	medium
7	54	51667.0	Berlin	M	1	small

In [36]:

```
# --- Solution ---
# cat_cols_s = small.select_dtypes(include=['object', 'category']).columns.tolist()
# small.dropna(subset=cat_cols_s)
```

6. Simple transformations¶

Four things you'll do a lot.

In [37]:

```
# Rename columns - useful for cleaning long auto-generated names
df_renamed = df.rename(columns={
    "sepal length (cm)": "sepal_len",
    "sepal width (cm)": "sepal_wid",
    "petal length (cm)": "petal_len",
    "petal width (cm)": "petal_wid",
})
df_renamed.head()
```

Out[37]:

	sepal_len	sepal_wid	petal_len	petal_wid	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

In [38]:

```
# Drop a column
small.drop(columns=["gender"]).head()
```

Out[38]:

	age	income	city	label	size
0	22	10734.0	Hamburg	0	large
1	58	20467.0	Munich	1	large
2	52	NaN	Munich	1	large
3	40	35256.0	Hamburg	0	small
4	40	39748.0	Munich	0	medium

In [39]:

```
# Add a new column derived from existing ones
small["age_squared"] = small["age"] ** 2
small.head()
```

Out[39]:

	age	income	city	gender	label	size	age_squared
0	22	10734.0	Hamburg	F	0	large	484
1	58	20467.0	Munich	M	1	large	3364
2	52	NaN	Munich	M	1	large	2704
3	40	35256.0	Hamburg	F	0	small	1600
4	40	39748.0	Munich	M	0	medium	1600

In [40]:

```
# Convert types
small["age"] = small["age"].astype(int)
small["income"] = small["income"].astype(float)
small.dtypes
```

Out[40]:

```
age          int64
income       float64
city         object
gender       object
label        int64
size         object
age_squared  int64
dtype: object
```

7. Splitting features (X) from labels (y)¶

Two equivalent patterns. Use whichever feels clearer.

In [41]:

```

# By name (clearest)
X = df.drop(columns=["target"])
y = df["target"]
print("X:", X.shape, " y:", y.shape)

X: (150, 4)    y: (150,)

In [42]:

# By position (works when the label is the last column)
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
print("X:", X.shape, " y:", y.shape)

X: (150, 4)    y: (150,)

```

Exercise 5a — Splitting features from labels¶

Using `df` (Iris), create:

- `X` — all columns **except** "target"
- `y` — just the "target" column

Print both shapes.

```

In [43]:

# Exercise 5a
X = df.drop(columns=['target']) # your code here
y = df['target'] # your code here
print('X:', X.shape, ' y:', y.shape)

X: (150, 4)    y: (150,)

In [44]:

# --- Solution ---
# X = df.drop(columns=['target'])
# y = df['target']
# print('X:', X.shape, ' y:', y.shape)

```

8. Pandas → scikit-learn → numpy → PyTorch¶

Your thesis pipeline will roughly be:

1. **Load** with pandas.
2. **Scale** numerical columns, **encode** categorical columns (scikit-learn).
3. Convert the result to a **numpy** array.
4. Convert to a **torch** tensor and feed it to the encoder.

That's the whole journey from raw CSV to PyTorch.

```

In [45]:

```

```

# Pandas → numpy
arr = X.to_numpy()
print(type(arr).__name__, arr.shape, arr.dtype)

ndarray (150, 4) float64

In [46]:

# A typical preprocessing block for tabular data
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Use the small synthetic dataset which has both numerical and categorical columns
X_small = small.drop(columns=["label", "age_squared"], errors="ignore")
y_small = small["label"]

num_cols = X_small.select_dtypes(include="number").columns.tolist()
cat_cols = X_small.select_dtypes(include=["object", "category"]).columns.tolist()

# Fill the one missing income value before scaling
X_small = X_small.fillna(X_small.mean(numeric_only=True))

preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_cols),
])

X_prep = preprocess.fit_transform(X_small)
print("Preprocessed shape:", X_prep.shape, " dtype:", X_prep.dtype)

Preprocessed shape: (8, 10) dtype: float64

```

Exercise 5b — Building the preprocessing pipeline from scratch¶

Using `small`:

1. `X_s = small.drop(columns=["label", "age_squared", "size"], errors="ignore")`
2. Find `num_cols_s` and `cat_cols_s` with `select_dtypes`
3. Fill missing numerical values with the column mean
4. Build a `ColumnTransformer` with `StandardScaler` for numerical, `OneHotEncoder` for categorical
5. `fit_transform(X_s)` and print the output shape

Expected: (8, ?) — the ? depends on the number of one-hot categories.

In [47]:

```

# Exercise 5b
from sklearn.compose import ColumnTransformer

```

```

from sklearn.preprocessing import StandardScaler, OneHotEncoder

# 1. Drop label and extra columns
X_s = small.drop(columns=['label', 'age_squared', 'size'], errors='ignore')

# 2. Find column types
num_cols_s = X_s.select_dtypes(include='number').columns.tolist() # your code here
cat_cols_s = X_s.select_dtypes(include=['object', 'category']).columns.tolist() # your code here

# 3. Fill missing numerical values
# your code here
X_s = X_s.fillna(X_s.mean(numeric_only=True))

# 4. Build ColumnTransformer
preprocess_s = ColumnTransformer([
    ('num', StandardScaler(), num_cols_s),
    ('cat', OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_cols_s)
]) # your code here

# 5. Fit-transform and print shape
X_s_prep = preprocess_s.fit_transform(X_s)
# your code here
print('Output shape:', X_s_prep.shape)

Output shape: (8, 7)

In [48]:

# --- Solution ---
# from sklearn.compose import ColumnTransformer
# from sklearn.preprocessing import StandardScaler, OneHotEncoder
#
# X_s = small.drop(columns=['label', 'age_squared', 'size'], errors='ignore')
# num_cols_s = X_s.select_dtypes(include='number').columns.tolist()
# cat_cols_s = X_s.select_dtypes(include=['object', 'category']).columns.tolist()
# X_s = X_s.fillna(X_s.mean(numeric_only=True))
# preprocess_s = ColumnTransformer([
#     ('num', StandardScaler(), num_cols_s),
#     ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_cols_s),
# ])
# X_s_prep = preprocess_s.fit_transform(X_s)
# print('Output shape:', X_s_prep.shape)

```

Exercise 5c — Capstone: full pipeline on breast_cancer ¶

Put everything together:

1. `from sklearn.datasets import load_breast_cancer` — load with `as_frame=True`

2. Inspect: `.info()`, check for missing values
3. Split into X and y
4. All features are numerical — build a `ColumnTransformer` with just `StandardScaler`
5. `fit_transform` → numpy array
6. Print the final shape — should be **(569, 30)**

In [49]:

```
# Exercise 5c - Capstone
# 1. Load
# your code here
from sklearn.datasets import load_breast_cancer
# 2. Inspect
# your code here

# 3. Split X / y
# your code here

# 4. Build ColumnTransformer (numerical only)
# your code here

# 5. Fit-transform
# your code here

# 6. Print shape
# your code here
```

In [50]:

```
# --- Solution ---
# from sklearn.datasets import load_breast_cancer
# from sklearn.compose import ColumnTransformer
# from sklearn.preprocessing import StandardScaler
#
# bc = load_breast_cancer(as_frame=True)
# df_bc = bc.frame
# print(df_bc.info())
#
# X_bc = df_bc.drop(columns=['target'])
# y_bc = df_bc['target']
#
# num_cols_bc = X_bc.select_dtypes(include='number').columns.tolist()
# preprocess_bc = ColumnTransformer([('num', StandardScaler(), num_cols_bc)])
#
# X_bc_prep = preprocess_bc.fit_transform(X_bc)
# print('Final shape:', X_bc_prep.shape) # (569, 30)
```

In [51]:

```
# (Later in your thesis) numpy → torch tensor for the encoder
# import torch
# X_tensor = torch.from_numpy(X_prep.astype(np.float32))
# print(X_tensor.shape, X_tensor.dtype)
```

9. Saving and loading results¶

You'll use this to store experiment results — one row per (dataset, augmentation, seed) combination, columns for NMI, ARI, silhouette, etc.

In [52]:

```
results = pd.DataFrame({
    "dataset":      ["iris",      "iris",      "adult"],
    "augmentation": ["masking",    "noise",     "masking"],
    "seed":         [0,           0,           0],
    "nmi":          [0.78,        0.71,        0.42],
    "ari":          [0.74,        0.68,        0.39],
})
results
```

Out[52]:

	dataset	augmentation	seed	nmi	ari
0	iris	masking	0	0.78	0.74
1	iris	noise	0	0.71	0.68
2	adult	masking	0	0.42	0.39

In [53]:

```
# Save (don't write the row index - it's just 0,1,2,...)
results.to_csv("results.csv", index=False)
```

```
# Load back
results_back = pd.read_csv("results.csv")
results_back
```

Out[53]:

	dataset	augmentation	seed	nmi	ari
0	iris	masking	0	0.78	0.74
1	iris	noise	0	0.71	0.68
2	adult	masking	0	0.42	0.39

Exercise 6a — Saving experiment results¶

Simulate 3 benchmark runs. Create a DataFrame with columns `dataset`, `augmentation`, `seed`, `nmi`, `ari` and 3 rows of your choice. Save to `"results_test.csv"` (no row index), reload it, and display it.

In [54]:

```
# Exercise 6a
results_ex = pd.DataFrame({
    'dataset':      ['iris', 'iris', 'adult'],    # your values here
    'augmentation': ['masking', 'noise', 'masking'],
    'seed':         [0, 0, 0],
    'nmi':          [0.78, 0.71, 0.42],
    'ari':          [0.74, 0.68, 0.39],
})
```

```
# Save to CSV
# your code here
results_ex.to_csv("results_test.csv", index=False)
# Reload and display
# your code here
resultsTest_back = pd.read_csv("results_test.csv")
resultsTest_back
```

Out[54]:

	dataset	augmentation	seed	nmi	ari
0	iris	masking	0	0.78	0.74
1	iris	noise	0	0.71	0.68
2	adult	masking	0	0.42	0.39

In [55]:

```
# --- Solution ---
# results_ex = pd.DataFrame({
#     'dataset':      ['iris',      'wine',      'breast_cancer'],
#     'augmentation': ['masking', 'noise',      'masking'],
#     'seed':         [0,          0,          0],
#     'nmi':          [0.78,       0.61,       0.45],
#     'ari':          [0.74,       0.58,       0.41],
# })
# results_ex.to_csv('results_test.csv', index=False)
# pd.read_csv('results_test.csv')
```

10. Things you don't need to learn (for *this* thesis)¶

Pandas is huge. For this thesis you can safely skip:

- `groupby` and aggregation — only needed if you later want to summarise the results table, and it's easy to pick up then.
- `merge` / `join` — database-style joins between two DataFrames.
- Time series and `resample`.
- `MultiIndex`.
- `pivot` / `melt` — long wide reshape.

If you ever need them, the official docs at pandas.pydata.org/docs are excellent. But none of these are on the critical path for your benchmark.

11. End-to-end mini-example¶

This is essentially the pattern your benchmark code will use: **load** → **inspect** → **split features/label** → **preprocess numerical and categorical separately** → **numpy array ready for the encoder**.

In [56]:

```
# 1. Load - Iris from sklearn, plus a fake categorical column for realism
from sklearn.datasets import load_iris
```

```
iris = load_iris(as_frame=True)
df = iris.frame.copy()
rng = np.random.default_rng(seed=0)
df["origin"] = rng.choice(["A", "B", "C"], size=len(df))
```

```
# 2. Inspect
print("Shape:", df.shape)
print("Missing values total:", df.isna().sum().sum())
df.head()
```

Shape: (150, 6)

Missing values total: 0

Out[56]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	origin
0	5.1	3.5	1.4	0.2	0	C
1	4.9	3.0	1.4	0.2	0	B
2	4.7	3.2	1.3	0.2	0	B
3	4.6	3.1	1.5	0.2	0	A
4	5.0	3.6	1.4	0.2	0	A

In [57]:

```

# 3. Split features and label
X = df.drop(columns=["target"])
y = df["target"]

# 4. Separate numerical and categorical
num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = X.select_dtypes(include=["object", "category"]).columns.tolist()
print("num_cols:", num_cols)
print("cat_cols:", cat_cols)

# 5. Preprocess
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_cols),
])
X_prep = preprocess.fit_transform(X)

print("X_prep shape:", X_prep.shape, " dtype:", X_prep.dtype)
print("y unique  :", np.unique(y))

num_cols: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
cat_cols: ['origin']
X_prep shape: (150, 7) dtype: float64
y unique  : [0 1 2]

```

That's the whole pipeline. From here in your thesis you will:

1. Wrap `X_prep` in a PyTorch `Dataset` / `DataLoader`.
2. Train the MLP encoder with a contrastive loss (augmentations come in here).
3. Run k-means on the resulting embeddings.
4. Score the clustering against `y` with NMI and ARI.

Three real-world tips that will save you time:

- Always run `df.info()` first on any new dataset — sizes, dtypes and missing values at a glance.
- For OpenML datasets, set `as_frame=True` so you get a pandas DataFrame directly.
- If a CSV throws weird parsing errors, try: `pd.read_csv(path, sep=";", encoding="latin-1", na_values=["?"])`. Those three arguments fix 80% of issues.

12. Quick reference card¶

The 20 lines of pandas you'll actually use in this thesis.

Task	Code
Load CSV	<code>pd.read_csv("file.csv")</code>
Load sklearn dataset	<code>load_iris(as_frame=True).frame</code>
Shape	<code>df.shape</code>
First rows	<code>df.head()</code>
Types + nulls	<code>df.info()</code>
Numerical stats	<code>df.describe()</code>
Missing per column	<code>df.isna().sum()</code>
Class balance	<code>df["target"].value_counts()</code>
Single column	<code>df["col"]</code>
Multiple columns	<code>df[["a", "b"]]</code>
Row by position	<code>df.iloc[0]</code>
Row by label	<code>df.loc[0]</code>
Boolean filter	<code>df[df["x"] > 0]</code>
Numerical only	<code>df.select_dtypes(include="number")</code>
Categorical only	<code>df.select_dtypes(include="object")</code>
Drop NaN	<code>df.dropna()</code>
Fill NaN	<code>df["x"].fillna(value)</code>
Drop column	<code>df.drop(columns=["x"])</code>
Rename	<code>df.rename(columns={"a": "b"})</code>
To numpy	<code>df.to_numpy()</code>
Save CSV	<code>df.to_csv("out.csv", index=False)</code>

That's the whole toolkit. Once these feel natural, you have everything you need from pandas for the thesis.